All posts

NanoGPT-Bench

Can coding agents do research?

Published May 19, 2026

At Intology, we create systems that automate research. As part of that mission, we develop benchmarks to measure AI systems' capability to perform research on long-horizon scientific problems.

Existing benchmarks in this direction lack important properties for clearly measuring AI R&D performance, such as having low contamination, substantial performance headroom, and a clean metric unconfounded by low-hanging fruit. To bridge this gap, we release a benchmark we find useful internally: NanoGPT-Bench - based on the popular GPT-2 pretraining speedrun challenge *NanoGPT Speedrun* - which measures an AI system's ability to perform open-ended frontier AI research.

In NanoGPT-Bench, agents work *fully autonomously* to recover historical progress in the NanoGPT speedrun competition [1]. Since 2024, human progress on *NanoGPT Speedrun* contributed to the development of the Muon optimizer [2] and leveraged cutting-edge developments in the field like Polar Express matrix multiplication [3], the NorMuon optimizer [4], and selective sign cautious weight decay [5]. This human progress provides a rich, competitive horizon to compare agent progress with.

Below, we describe the evaluation setup and baseline results for NanoGPT-Bench. **We ran Claude Code, Codex, and Autoresearch, and find all agents struggle to recover significant pretraining speedup given substantial compute, achieving less than 10% of the speedup achieved by human solutions.** In contrast with humans, these coding agents mostly perform hyperparameter tuning and largely fail at algorithmic research.

GitHub: https://github.com/IntologyAI/NanoGPT-Bench

Introducing NanoGPT-Bench

NanoGPT-Bench evaluates agents on the *NanoGPT Speedrun* environment at a fixed starting point. We use the human world record as of September 3rd, 2025, which is the first record after current frontier models' knowledge cutoffs. Agents work fully autonomously, with no human intervention and no internet access. Submissions go through a `submit` command that checks competition rules via an LLM judge and retimes the candidate across ten runs to confirm a statistically significant speedup, mirroring the original *NanoGPT Speedrun* review process. We compare agent trajectories against the 33 human world records set between September 3rd, 2025 and January 19th, 2026, corresponding to roughly five months of community progress.[1] Full environment details, baseline configurations, and validity checks are in the Appendix.

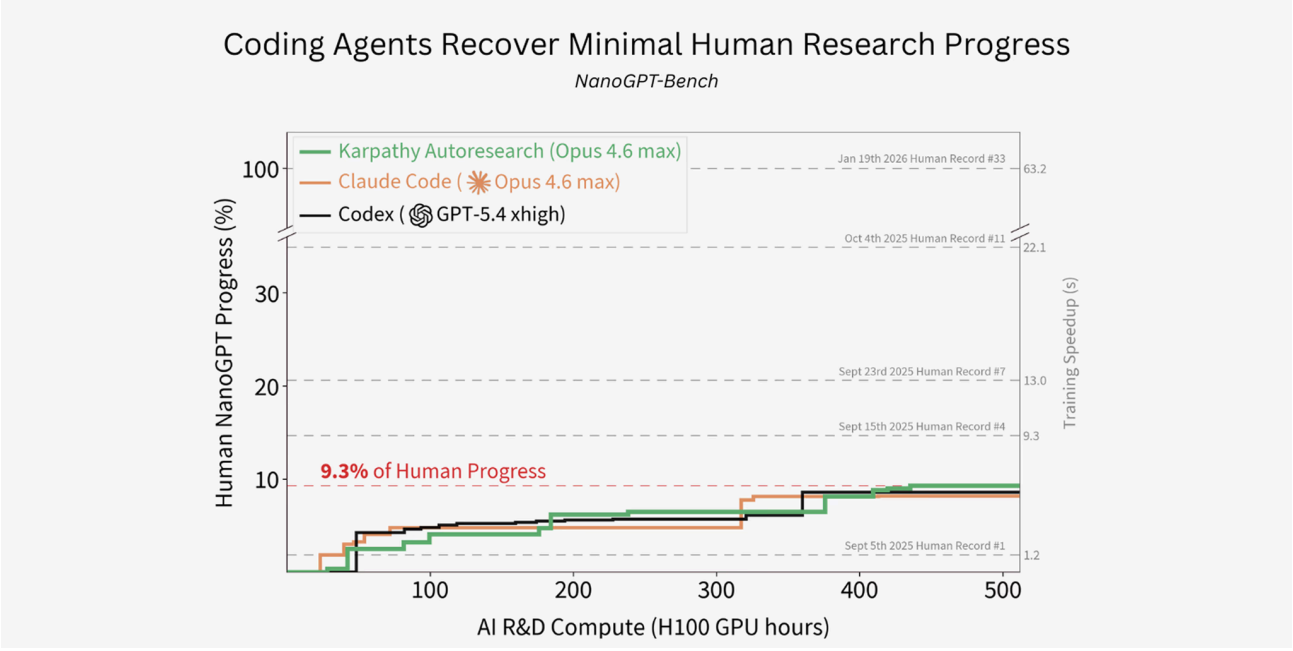

Figure 1. Best training time achieved by agents over a fixed H100 GPU hour budget, starting from the human world record as of September 3rd, 2025. Progress is shown as a percentage of the speedup achieved by the January 19th, 2026 human world record. All coding agent baselines were given a budget of 512 H100 GPU hours each, and recover less than 10% of the human world record progress.

We show baseline coding agents' performance trajectories on NanoGPT-Bench in Figure 1. Agents were run with a compute budget of 512 H100 GPU hours, running for up to a week of wall-clock time. Claude ran training on 455 different variants, Autoresearch on 321, and Codex on 399 different training variants. Given this substantial compute budget, coding agents recover less than 10% of human world record progress over the September 2025 - January 2026 human world record horizon we consider. The Autoresearch and "vanilla" Claude Code (Opus 4.6 Max) variants achieve 9.3% and 8.2% performance, respectively; Codex (GPT 5.4 xhigh) achieves 8.6%.

These are larger speedups than the first human record (labelled "Human Record #1 in Figure 1), which corresponds to 2 days of human record progression. The Autoresearch baseline slightly outperforms the second human record, corresponding to roughly 7 days of human record progression. Nonetheless, these speedups fall far short of the cumulative 63.2 seconds of speedup achieved by human world records within the 5 months after September 2025.

Coding Agents Focus on Hyperparameter Tuning

Each baseline in NanoGPT-Bench produces a long trajectory of reasoning, tool calls, and submissions. To obtain a more granular view of agent behavior, we extract and perform classification over "spans", defined as sequences of reasoning and tool call IO between consecutive `submit` invocations. For each span, we also extract the diff between the submission code on either end of the span. We operationalize a simple majority-vote multi-class LLM classifier via GPT-5.4 medium over the taxonomy described in the Appendix; the classifier takes as input each (span, diff) tuple into the LLM, and produces a set of classes from the taxonomy that describe the submission. We further attribute each class to a portion of the submission diff via an LLM, and designate the class corresponding to the largest diff portion as the "primary" class.

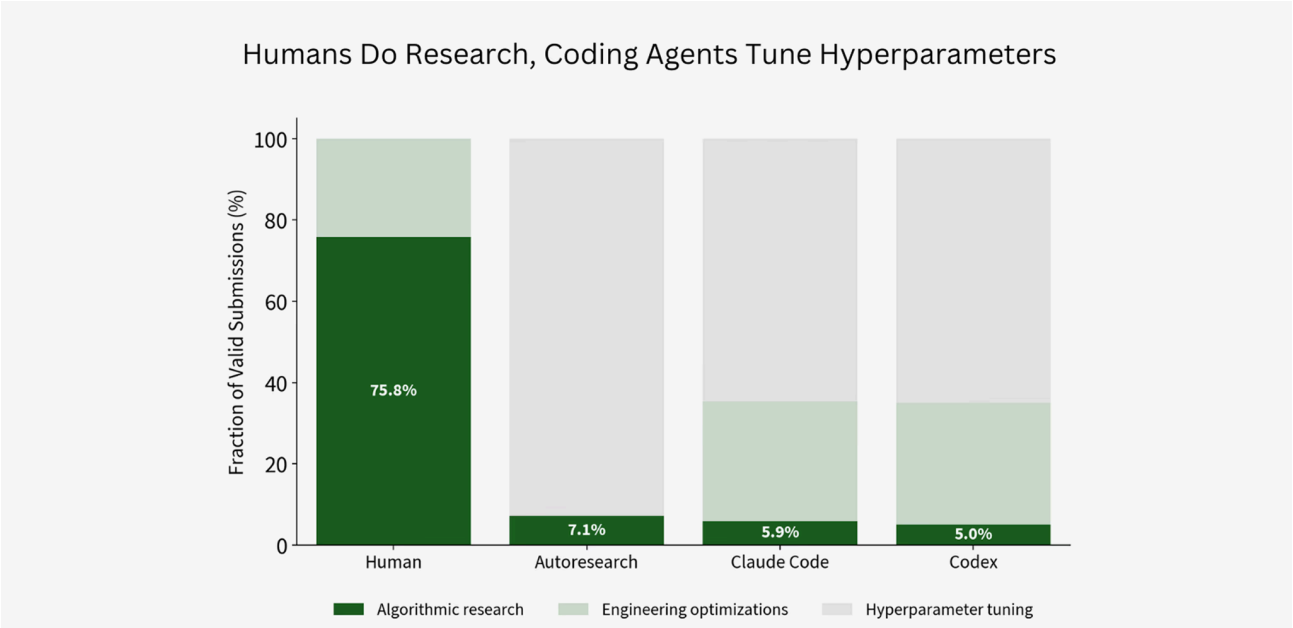

Figure 2. Classification of human records and baseline submissions over several categories: algorithmic research, engineering optimizations, and hyperparameter tuning. Human world records include algorithmic research changes roughly 77% of the time, while agent submissions include algorithmic changes less than 10% of the time. Agent submissions predominantly perform hyperparameter tuning.

We show the distribution of primary submission classifications in Figure 2. Coding agents do substantially *less* algorithmic research than humans, which may be a contributing factor to their low performance. They spend the majority of their execution performing hyperparameter tuning, often changing simple values like window size scheduling, learning rate, or the total training steps. By contrast, successful human records introduce algorithmic changes roughly 75.8% of the time. When human records do make hyperparameter changes, they are always coupled with additional algorithmic changes or engineering optimizations.

In Figure 2, we consider valid submissions made by humans and agents, though we also consider valid submissions made by agents that passed the p-value requirements but did not outperform the prior best solution, while we only have access to successful human submissions. That being said, we interpret the observed human solution distribution as a representative sample of what constitutes successful submissions, which serves as a useful reference point from which we draw our conclusions.

Coding Agents Rarely Attempt Research

While categorization of high-level submissions provides insight into the baselines' coverage of relevant solution types, we find it useful to further breakdown the baselines' agent traces to analyze their *behavior*. Specifically, we use the combination of spans and diffs between submissions to detect the following for agents' engagement with algorithmic research:

- Did an agent mention algorithmic change at all?
- Did an agent mention an algorithmic change, but not implement it?
- Did an agent implement an algorithmic change, but fail to obtain a speedup?
- Did an agent implement an algorithmic change, and obtain a speedup?

We show this distribution of algorithmic research engagement in Figure 3. Codex primarily avoids algorithmic research altogether. Claude Code and Autoresearch both reason more about algorithmic research, but still dodge implementation.[2] Given the baselines' low performance in Figure 1, and the stark contrast with humans' success with algorithmic research in Figure 2, we believe the baselines' lack of successful algorithmic research may be a major contributor to their unremarkable performance.

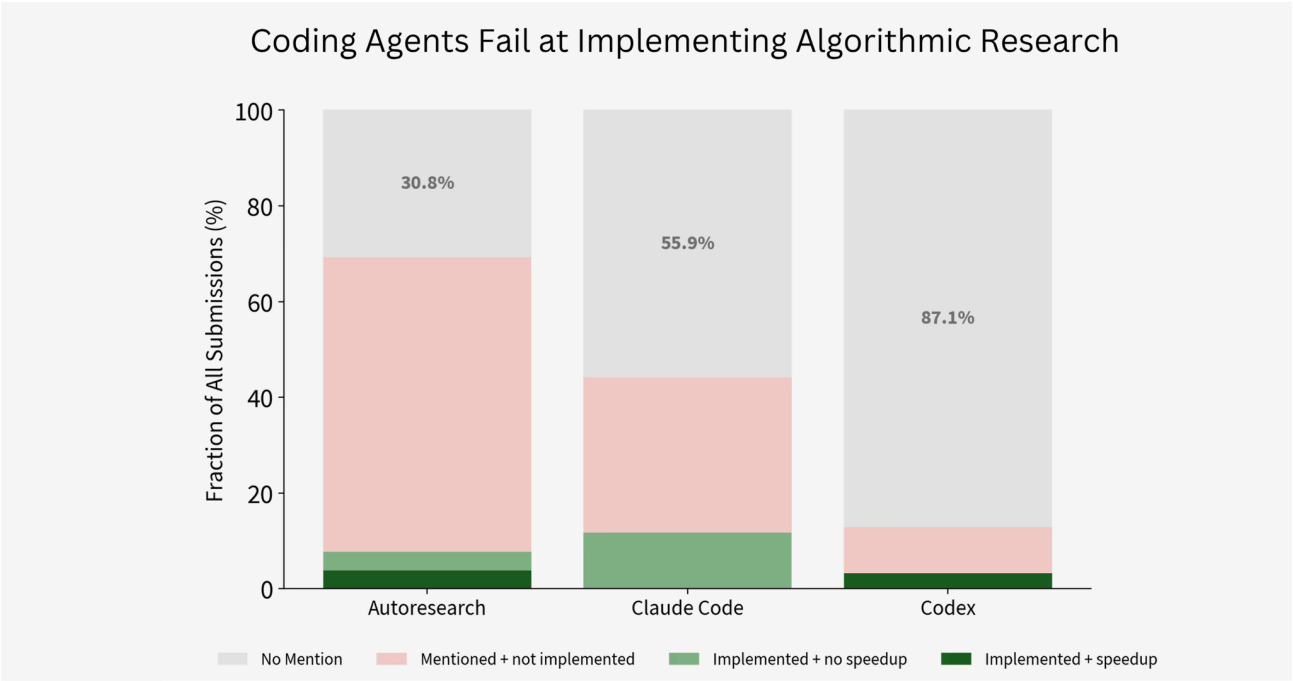

Figure 3. Rates of agent traces that discuss algorithmic research, classified by performance of resulting submissions. A new fastest canonical submission is defined as a submission that passes the submission validation LLM judge, and achieves a faster mean training time than the previous best submission at the time of submission. The baseline coding agents are largely unsuccessful at performing algorithmic research.

NanoGPT-Bench Rigorously Evaluates Research Capabilities

Environment	Human Time Horizon	Optimized Initialization	Research-Oriented	Autonomous
BENCHMARKS				
NanoGPT-Bench	H D W M	✓	✓	✓
NanoGPT-Speedrunner	H D W M	✓	⚠	⚠
RE-Bench	H D W M	✗	✓	✓
PaperBench	H D W M	✗	⚠	✓
MLE-Bench	H D W M	✗	✓	✓
ProgramBench	H D W M	✗	✗	✓
MLGym	—	✗	✓	✓
AUTONOMOUS RESEARCH RUNS				
Prime Intellect NanoGPT Run	H D W M	✗	✓	✗
Claude C Compiler	—	✗	✗	✓
SkyPilot AutoResearch	—	✗	✓	✓

Table 1. Comparison of the NanoGPT-Bench environment to prior benchmarks and evaluations. Human time horizon is the estimated human time it would take to fully saturate the benchmark. Optimized starting point checks if the initial implementation given to the agents has already undergone human optimization. Research oriented distinguishes between open-ended development tasks and more constrained SWE-focused tasks. Finally, autonomous indicates if results are obtained without human intervention and guidance over the course of the run.

We've found the following properties important for autonomous research evaluation:

1. Providing an open-ended problem to solve
2. Initializing from a strong prior solution
3. Including a long time horizon human reference comparison

Each of these properties enables an important component of the evaluation. Open-ended research is necessary to test models' ability to come up with ideas themselves, not just follow human instructions. A long-horizon human comparison highlights models' current deficiencies and suggests ways to improve them. Finally, an optimized initialization prevents the measurement from being confounded by low-hanging fruit, incentivizing progress through non-trivial approaches.

Many prior benchmarks have focused on shorter horizon tasks with a less optimized initialization, while previous publicly documented large-scale autonomous research attempts have largely lacked rigorous evaluation setups (see further discussion in Appendix). By contrast, NanoGPT-Bench contains an automatic validation oracle, optimized human record initialization, and rich history of human records to compare against.

Where To Go From Here

We're excited to push the boundaries of automated research and consider evaluations a core part of that mission.

We think evaluations of autonomous AI R&D should measure ability to push the research frontier with novel solutions, rather than merely apply known solutions to easy problems. We've found NanoGPT-Bench a useful proxy for real-world research problems, where prior work has often optimized away low-hanging fruit.

The NanoGPT-Bench setting highlights the disparity between current coding agents and the research humans are able to conduct over the course of months. The top performing coding agent only recovered 9.3% of the human progress within 5 months and disproportionally engaged in hyperparameter and engineering optimizations, suggesting real gaps between the more fundamental algorithmic research performed by humans and the classes of solutions explored by coding agents.

Use of NanoGPT-Bench has helped enable the development of our automated research system, Locus, which in January set a world record on the human *NanoGPT Speedrun* leaderboard by designing a new kernel algorithm. Going forward, we are excited to continue pushing on both autonomous research and the evaluations to measure it.

Acknowledgements

We thank Keller Jordan for creating the human speedrun setting and Larry Dial for discussion and maintaining the *NanoGPT Speedrun* leaderboard.

We'd like to give a shoutout to Soren Dunn, on our team, for taking point on this project!

Appendix

Prior Work

There are a plethora of prior benchmarks and evaluations focused on evaluating agents' research abilities and/or general long-horizon coding abilities. On the software engineering side, Anyscale and Anthropic have both performed large-scale runs testing their agents' capabilities in building large software projects - specifically in developing a web browser and building a C compiler, respectively. Popularized by Karpathy's autoresearch , there have also been attempts to similarly apply coding agents to automate research, running coding agents on research tasks over the course of days or weeks. Though allowing larger-scale compute evaluations than typical benchmarks, many of these attempts suffer from inconsistent settings, varying levels of human intervention, and lack of consistent validity and/or overfitting tests.

On the other hand, separate drawbacks limit the usefulness of various academic benchmarks in evaluating long-horizon autonomous research capabilities. Traditional benchmarks focus on well-defined, short-horizon tasks [8, 9], whereas evaluating research agents requires environments that incentivize exploration and experimentation. Even for longer-horizon tasks many focus more on software engineering tasks [10] or lack a point of human comparison [11, 12]. Human expert comparisons in the long horizon setting are particularly difficult to obtain due to the inherent length of time and resource constraints such evaluations take to get. The *NanoGPT Speedrun* is a particularly useful environment therefore due to its long history of expert human submissions. Other benchmarks start with simplistic or noop solutions [9, 13, 14] which inflate model performance due to their unoptimized starting point.[3]

Metrics

In the original *NanoGPT Speedrun*, the metric is the wall-clock time to train a model to a validation loss of 3.28 on a shard of the FineWeb dataset on a single 8xH100 node, subject to additional robustness constraints clarified below. Lower training time is better.

In NanoGPT-Bench, we provide baseline agents with an existing human record from *NanoGPT Speedrun,* an environment with fixed dependencies, and a fixed compute budget for performing experiments. Given this budget and initialization, the NanoGPT-Bench metric is the largest valid speedup achieved relative to the starting human record. Higher total valid speedup is better.

Environment

Initialization and human reference. As of this blog post, the *NanoGPT Speedrun* has had 80 world record submissions over the course of nearly 2 years. To enable a faithful comparison between baseline agents and human records in NanoGPT-Bench, we identify the first human record after recent frontier LLMs' knowledge cutoff dates to use as initialization for baselines, which corresponds to September 3rd, 2025 in the evaluations we perform.[4] We then extract the longest possible trajectory of human reference records which are runnable using the same fixed dependencies as the September 3rd record, which ends at January 19th, 2026. This yields 33 human records; we retime these records with the initial fixed set of dependencies to ensure reproducibility in our comparisons. We also use this set of human records as an additional performance reference for baselines as described in the Evaluation section. Additionally, internet access is disallowed for agents.

Submission validation. The environment includes a `submit` command that agents can invoke to make official submissions. This `submit` script first checks a candidate submission against an LLM judge to ensure it adheres to the competition rules (analogous to the review humans received when submitting PRs to *NanoGPT Speedrun*).[5]

Compute budget. We define a compute budget for total experimentation, measured in cumulative GPU hours. Total cumulative GPU hours are the combined hours an agents' jobs spent running on the GPUs. Evaluation is performed by running baselines until they fully saturate this compute budget. Note that NanoGPT-Bench is parametrized by the initial conditions and compute budget, and the setup can be adapted to avoid future contamination.

Baselines & Evaluation

We evaluate three recent frontier coding agent baselines: Codex (GPT-5.4 xhigh), Claude Code (Opus 4.6 Max), and Opus 4.6 Max within a custom scaffold around Claude Code that reuses prompting from Andrej Karpathy's Autoresearch harness.[6]

Each baseline is given a 512 H100-hour compute budget. All harnesses are augmented with custom "goal-mode" or "Ralph Wiggum" style loops preventing early terminations. Specifically, if agents attempt to terminate before fully consuming the 512 H100-hour compute budget, they are automatically resumed in the same session with instructions to continue to improve their implementations. As mentioned, we perform all evaluations with no manual intervention in any capacity. Finally, all agents are given the same initialization described in the previous section.

In addition to the final speedup achieved, we collect the set of valid submissions along each baseline coding agent's trajectory, recording the frontier of performance over consumed H100 compute. Because we hold total H100 compute equal, these submissions can be faithfully compared across baselines as compute varies. We also find it informative to compare agents' performance trajectories with the human record reference between September 3, 2025 and January 19th, 2026. In our results, GPU compute cost dominates LLM token cost for these baselines, so we report only GPU compute usage for simplicity.

NanoGPT-Bench Taxonomy

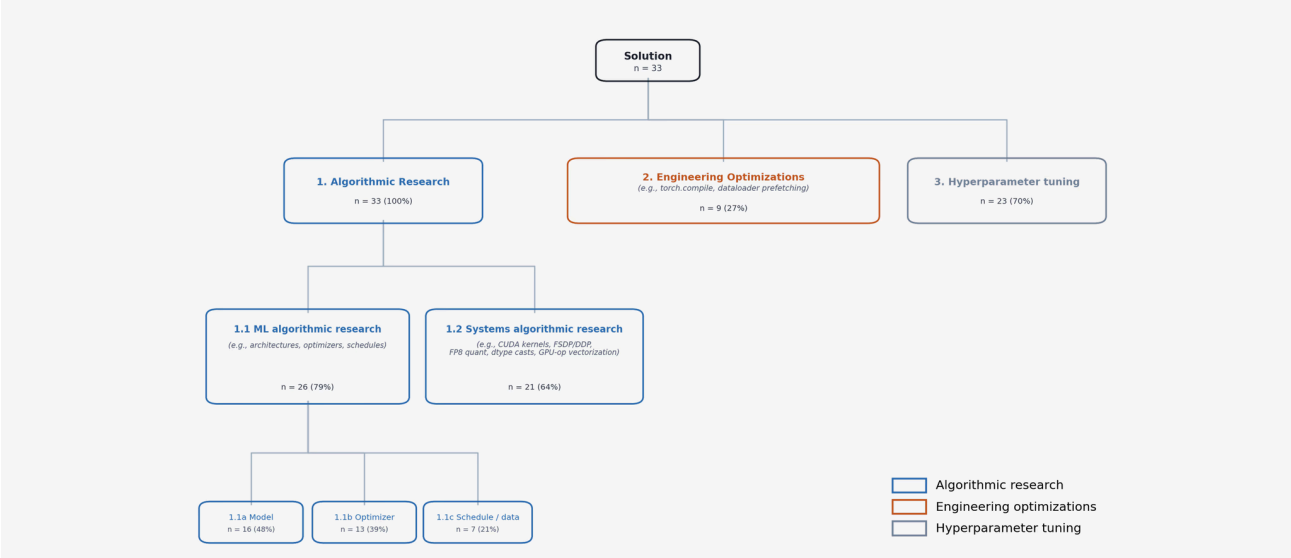

Figure 4. A hierarchical classification of agent and human solutions on the modded-nanogpt speedrun benchmark. The categories are grounded inductively in the 33 merged human PRs (Sept 2025 – Jan 2026). Sums exceed 33 due to multi-category tagging.

We define the following taxonomy grounded in the existing human world record submissions in the *NanoGPT Speedrun:*

1. **Algorithmic Research:** A change whose diff acts on a named algorithmic artifact. The artifact is either ML-side (a model architecture, an optimizer update rule, a schedule, a sampling strategy) or systems-side (a kernel, a distributed-comm algorithm, a parallelism strategy, a quantization scheme, GPU-side computation of the training math).
    1. **ML algorithmic research** (e.g., new architectures, optimizers, schedules): A change whose named artifact is a piece of ML mathematics: a model architecture or sub-component, a loss / objective, an optimizer update rule, a regularization rule, a schedule shape, or a sampling / curriculum strategy.
        1. **Model.** Examples: adding or removing a layer, restructuring skip connections, modifying the attention mechanism (positional encoding, head structure, key/value projections), changing the embedding or output head (tying, hashing, multi-token prediction), changing the loss/objective (auxiliary losses, prediction targets), changing the residual stream between layers.
        2. **Optimizer.** Examples: introducing a new update rule (Muon variants, sign-method changes, polar-decomposition approximations), introducing a new regularization rule (cautious weight decay), special-case handling of parameter subsets, gating mechanisms inside the update.
        3. **Schedule / data.** Examples: batch-size schedules, learning-rate schedule shape (not values), curriculum / data ordering, data augmentation, sampling-rule changes. Changes which do nothing more than modify numerical constants or lists/tuples of numerical constants in the code do not constitute schedule/data changes.
    2. **Systems algorithmic research**: A change whose diff acts on the existing algorithmic computation of the training math, usually making existing operations more efficient (e.g., CUDA kernels, FSDP/DDP-style comm, FP8 quant, dtype casts, GPU-op vectorization). The mathematical specification of training is unchanged (modulo small tolerated numerical drift); the algorithm or implementation used to compute that math on the device is the subject of the diff.
2. **Engineering Optimizations**: A change whose diff reorganizes host-side code without changing what the GPU computes. Examples: code reorganization to enable compilation, applying `@torch.compile` to existing paths, dataloader prefetching, async data-loading plumbing, batching-layout changes, dispatch refactors, param-table refactors.
3. **Hyperparameter tuning**: A change to the values of free parameters of the training function without changing its structure or implementation (e.g., learning rate, window size schedule, total training steps). This includes all changes which do nothing more than modify numerical constants or lists/tuples of numerical constants in the code.

Additional Validity Checks

In order to ensure the validity of the environment, we performed additional contamination and overfitting checks on the agents and their solutions.

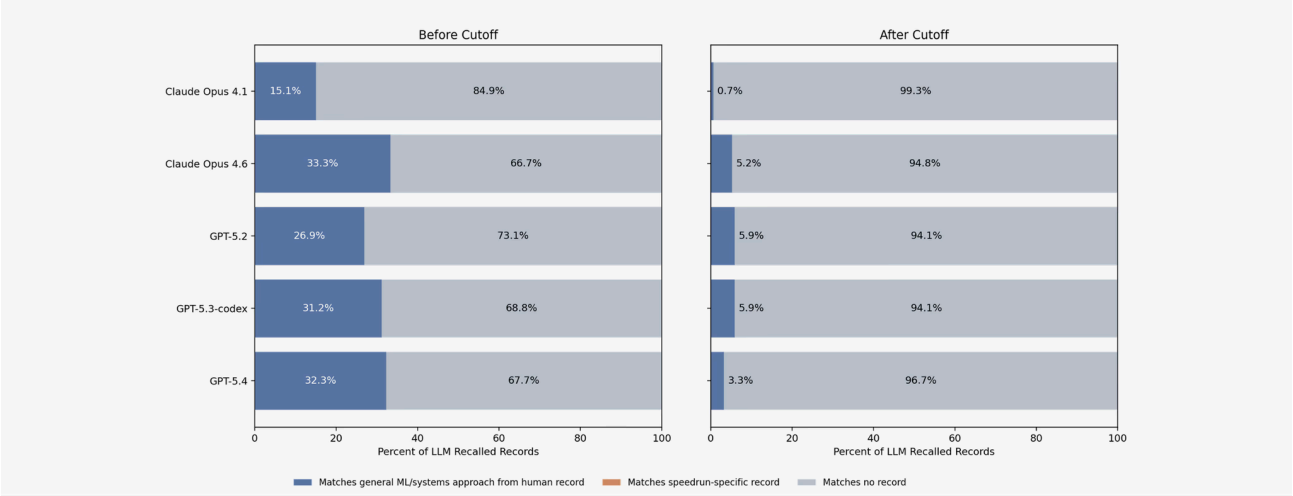

Figure 5. Contamination rates across model families both before and after the baseline September 3rd 2025 record. Models are required to describe all optimizations present in the current NanoGPT world record without access to external resources. Listed improvements are subsequently classified by an LLM judge in pairwise comparisons into not matching, matching only in general or approach, or matching in specific approach compared to existing human records. No records either before or after the September 3rd start date are described in detail by any model but the rate of general techniques being described based on pretraining knowledge is significantly higher from the earlier, pre start date records.

For contamination, we prompted models from the GPT and Claude model families to describe all optimizations present in the current NanoGPT world record without access to external resources. Listed improvements are subsequently classified by an LLM judge via pairwise comparisons into the following classes:

- not matching
- matching only in general or approach
- matching in specific approach compared to existing human records.

No records either before or after the September 3rd start date are classified as matching in specific approach for any model. However, 15.1% to 33.3% of solutions elicited from models matched the general approach of human records from before the September 3rd start date. For human records after September 3rd, this rate drops to between 0.7% and 5.9%, again varying based on model used. These results indicate that there is no obvious contamination in the models' parametrized knowledge for the post-September 3rd human records we compare models' submissions to. Additionally, the discrepancy between records before and after the stated model cutoff dates are indicative of either earlier human records simply incorporating more general, well-known ML techniques and/or models displaying some knowledge of the pre-September 3rd records.

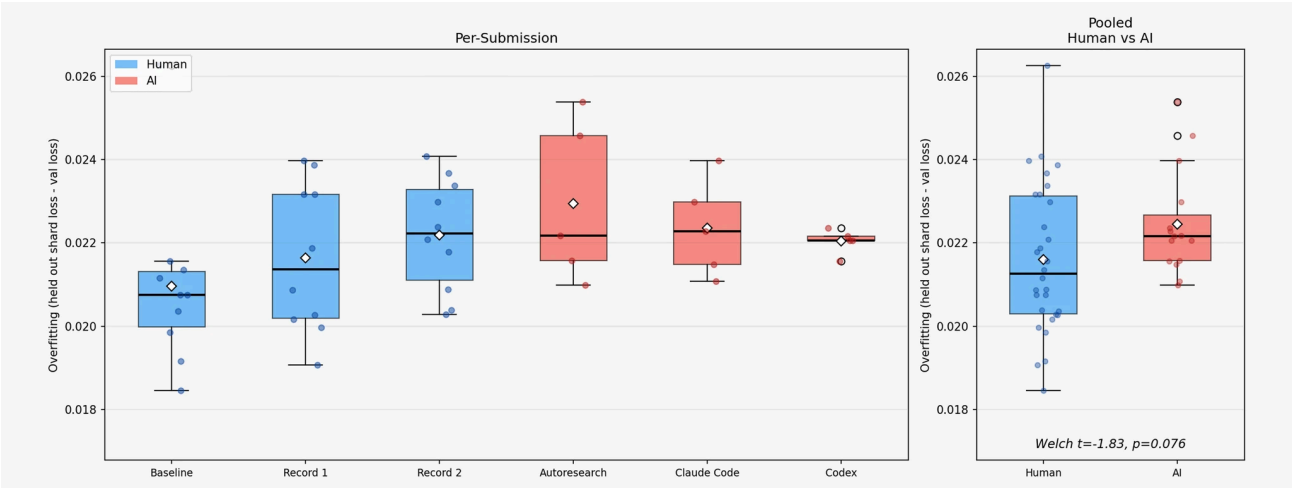

Figure 6. Distribution of difference in final validation loss of the best performing agent records and initial human records on held-out 100m token validation shard from the FineWeb dataset and the validation loss on the canonical validation set for the NanoGPT speedrun. Higher difference corresponds to higher overfitting on the canonical validation shard. Agents have a median validation increase slightly higher than the median human validation loss increase but not statistically significantly higher (p-value >0.05).

Even though each individual submission (made either by humans or LLMs) on the *NanoGPT Speedrun* leaderboard is screened for overfitting on the validation set, higher-order optimization against the validation tokens still occurs. Additionally, since agents tend to make submissions via the oracle `submit` command much more often than humans submit to the real *NanoGPT Speedrun* leaderboard, a potential concern is that agents exhibit higher overfitting to the validation tokens than human records. To gauge this effect, we compared the validation loss of submissions on a separate held-out set of 100 million FineWeb validation tokens to the original set of 10 million FineWeb validation tokens; the latter of which is used for canonical evaluation.[7]

Both human and agent records exhibit higher loss on the held-out 100 million token validation set than the original validation set. This is expected since the humans have been optimizing performance on the same validation set over an extended period of time and agents' solutions are initialized on top of a previous human record. To test the agents' solutions for overfitting the more important metric to look at is the difference between the validation loss increase for the humans' solutions compared to the agents'. We computed this validation loss increase for the September 3rd, September 5th, and September 10th human records as well as the highest-performing valid submissions from each baseline coding agent. As seen in figure 6, both humans' and agents' submissions have approximately the same increase in validation loss across these sets, with the agents' solutions having a slightly

higher mean validation loss but not statistically significantly so. From this, we observe that despite agents' solutions performing a large amount of hyperparameter tuning, they seem to be exhibiting minimal increased overfitting compared to human solutions.

Footnotes

[1] "Human" record #31 was actually achieved by Locus, our Artificial Scientist on January 16th 2026. Locus implemented a fused triton kernel for the softcapped multi-token prediction cross entropy step. Several future records by humans have built further on the kernel.

[2] For example, Autoresearch considered reducing the number of value embeddings from 3 to 2 17 separate times throughout its trajectory but never actually implemented this change. As a specific example of this reasoning:

"Reducing value embeddings from 3 to 2 would save a bit of computation, but the real issue is that layer 2 would lose its value embedding entirely, which could hurt convergence — and layer 9 already has no attention anyway so that embedding is wasted. The time savings wouldn't justify the convergence risk."

[3] The most similar previous setting to NanoGPT bench is the Automated LLM Speedrunning benchmark from Facebook. This benchmark used pairs of records from the very earliest NanoGPT Speedrun world records to see if agents were able to reproduce the human implementations given pseudocode, text descriptions, and/or mini papers of the records. They found that the top-performing model/scaffold at the time was able to recover a single world record worth of speedup 5.6% of the time without hints and even given the pseudocode and full text description of the human change was only able to recover a single human record at most 22.2% of the time. Additionally most of the models tested already had knowledge cutoff dates after the publication of some of these records. Todays' agents consistently recover one or more records worth of improvements without hints - necessitating the much longer horizon, unguided, and less contaminated setting we propose.

[4] This record start date corresponds to immediately after Flash-Attention 3 [6] was added to the record.

[5] The LLM judge is given the contents of the FA3 human record models were initialized with as well as the candidate `train_gpt.py` code the LLM is submitting for evaluation. The judge must respond with a boolean of 1 or 0 corresponding to if it rates the candidate solution is cheating as per the provided rules. Specifically the judge is told to monitor for:

```
moving work that the canonical record performs inside the measured path into setup, caching, preprocessing, w
precomputing, reusing, or skipping work that the canonical record still performs inside the measured path
changing the evaluated data, masking, token selection, normalization, aggregation, validation window, or eval
other code changes that make the candidate's val loss no longer directly comparable to the canonical record's
```

Additionally each agent is given a RULES.md in their folder with the following additional directives:

```
Ensure any modifications keep validation loss fully comparable to prior versions.
The number of torch.compile call sites may change if needed, but do not introduce new torch.compile flags or
Do not change the current inductor and compile-related flag setup. torch._dynamo.config.recompile_limit is se
Do not change the underlying data streams or next-token objective. Training data comes from fineweb_train_*.b
Do not change the validation data pipeline. Validation must continue to read fineweb_val_*.bin, keep val_toke
Preserve the timing methodology used for scoring: 30 warmup steps, restore the initial model and optimizer st
Do not optimize print statements.
Ensure that the full warmup from an empty cache stays under 10 minutes.
Do not brute force tune parameters to the validation set - make sure changes made to hyperparameters are prin
Any command that touches the GPU must be launched through submit or torchrun
```

The candidate submission is then retimed across a set of ten runs to ensure it is a statistically significant improvement [The requirement for a statistically significant improvement is that over the 10 runs it achieves a validation loss under 3.28 with a p-value under 0.01 and achieve a mean training time under the previous best achieved training time], which is also a submission requirement in the original NanoGPT Speedrun. [We found that this submission setup lets agents validate real improvements during long autonomous runs without need for human-in-the-loop validation. Before providing this verifier we found that models would often overestimate their progress and interpret variance in submissions' runtime as legitimate improvement.

[6] We made minimal tweaks to the Autoresearch harness to provide instructions on the problem being solved as well as how to use the environment's custom submit command.

[7] 100 million tokens were used rather than a separate set of 10 million tokens, matching the record's original number of validation tokens, due to previous evidence indicating high amounts of variance in validation loss between separate 10 million validation sets.

References

[1] Keller Jordan and contributors. "modded-nanogpt." GitHub repository. <https://github.com/KellerJordan/modded-nanogpt>

[2] Keller Jordan. "Muon: An optimizer for hidden layers in neural networks." Keller Jordan blog. <https://kellerjordan.github.io/posts/muon/>

[3] Noah Amsel, David Persson, Christopher Musco, and Robert M. Gower. "The Polar Express: Optimal Matrix Sign Methods and Their Application to the Muon Algorithm." arXiv:2505.16932. <https://arxiv.org/abs/2505.16932>

[4] Zichong Li, Liming Liu, Chen Liang, Weizhu Chen, and Tuo Zhao. "NorMuon: Making Muon more efficient and scalable." arXiv:2510.05491. <https://arxiv.org/abs/2510.05491>

[5] Lizhang Chen, Jonathan Li, Kaizhao Liang, Baiyu Su, Cong Xie, Nuo Wang Pierse, Chen Liang, Ni Lao, and Qiang Liu. "Cautious Weight Decay." arXiv:2510.12402. <https://arxiv.org/abs/2510.12402>

[6] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. "FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision." arXiv:2407.08608. <https://arxiv.org/abs/2407.08608>

[7] Intology AI. "New WR: Fused Softcapped Cross Entropy Kernel (-0.9s)." Pull Request #199, `KellerJordan/modded-nanogpt`, January 2026. <https://github.com/KellerJordan/modded-nanogpt/pull/199>

[8] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. "SWE-bench: Can Language Models Resolve Real-world Github Issues?" International Conference on Learning Representations, 2024. <https://openreview.net/forum?id=VTF8yNQM66>

[9] Hjalmar Wijk, Tao Roa Lin, Joel Becker, Sami Jawhar, Neev Parikh, Thomas Broadley, Lawrence Chan, Michael Chen, Joshua M. Clymer, Jai Dhyani, Elena Ericheva, Katharyn Garcia, Brian Goodrich, Nikola Jurkovic, Megan Kinniment, Aron Lajko, Seraphina Nix, Lucas Jun Koba Sato, William Saunders, Maksym Taran, Ben West, and Elizabeth Barnes. "RE-Bench: Evaluating Frontier AI R&D Capabilities of Language Model Agents against Human Experts." Proceedings of the 42nd International Conference on Machine Learning, PMLR 267:66772-66832, 2025. <https://proceedings.mlr.press/v267/wijk25a.html>

[10] Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, Johannes Heidecke, Amelia Glaese, and Tejal Patwardhan. "PaperBench: Evaluating AI's Ability to Replicate AI Research." Proceedings of the 42nd International Conference on Machine Learning, PMLR 267:56843-56873, 2025. <https://proceedings.mlr.press/v267/starace25a.html>

[11] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Mądry. "MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering." International Conference on Learning Representations, 2025; arXiv:2410.07095. <https://arxiv.org/abs/2410.07095>

[12] Deepak Nathani, Lovish Madaan, Nicholas Roberts, Nikolay Bashlykov, Ajay Menon, Vincent Moens, Amar Budhiraja, Despoina Magka, Vladislav Vorotilov, Gaurav Chaurasia, Dieuwke Hupkes, Ricardo Silveira Cabral, Tatiana Shavrina, Jakob Foerster, Yoram Bachrach, William Yang Wang, and Roberta Raileanu. "MLGym: A New Framework and Benchmark for Advancing AI Research Agents." arXiv:2502.14499, 2025. <https://arxiv.org/abs/2502.14499>

[13] Evan Chu, Rajan Agarwal, Abishek Thangamuthu, Brendan Graham, Justus Mattern, Freeman Jiang, Paul Cento, Swarnim Jain, Mersad Abbasi, Mohammad Hossein Rezaei, George Wang, Alex Zhang, Simon Guo, Karina Nguyen, Arash Bidgoli, Aditya Dalmia, Apoorv Dankar, Ashrut Vaddela, Calvin Chen, Keshav Kumar, Kushagra Vaish, Navid Pour, Rishyanth Kondra, Sagar Badiyani, Sidharth Giri, Snagnik Das, Soham Gaikwad, Syed Shah, Vagish Dilawari, and Vishal Agarwal. "FrontierSWE." Proximal Blog, April 2026. <https://www.frontierswe.com/blog>

[14] John Yang, Kilian Lieret, Jeffrey Ma, Parth Thakkar, Dmitrii Pedchenko, Sten Sootla, Emily McMilin, Pengcheng Yin, Rui Hou, Gabriel Synnaeve, Diyi Yang, and Ofir Press. "ProgramBench: Can Language Models Rebuild Programs From Scratch?" arXiv:2605.03546, 2026. <https://arxiv.org/abs/2605.03546>

[15] Bingchen Zhao, Despoina Magka, Minqi Jiang, Xian Li, Roberta Raileanu, Tatiana Shavrina, Jean-Christophe Gagnon-Audet, Kelvin Niu, Shagun Sodhani, Michael Shvartsman, Andrei Lupu, Alisia Lupidi, Edan Toledo, Karen Hambardzumyan, Martin Josifoski, Thomas Foster, Lucia Cipolina-Kun, Abhishek Charnalia, Derek Dunfield, Alexander H. Miller, Oisin Mac Aodha, Jakob Foerster, and Yoram Bachrach. "The Automated LLM Speedrunning Benchmark: Reproducing NanoGPT Improvements." NeurIPS 2025 Datasets and Benchmarks Track; arXiv:2506.22419. <https://arxiv.org/abs/2506.22419>

[16] Cursor. "Towards self-driving codebases." Cursor Blog. <https://cursor.com/blog/self-driving-codebases>

[17] Nicholas Carlini. "Building a C compiler with a team of parallel Claudes." Anthropic Engineering, February 5, 2026. <https://www.anthropic.com/engineering/building-c-compiler>

[18] Andrej Karpathy. "autoresearch." GitHub repository. <https://github.com/karpathy/autoresearch>

[19] Anthropic. "Ralph Wiggum Plugin." Claude Code GitHub repository. <https://raw.githubusercontent.com/anthropics/claude-code/main/plugins/ralph-wiggum/README.md>